第 12 节 实操任务 2：编写 Eval 评估测试

> 目标：pytest tests/eval_test.py 通过 3+ 个评估用例
> 目标文件：`tests/eval_test.py`

背景

AI 系统的输出不确定——同一个输入，每次调用可能得到不同的输出。传统的 `assert output == expected` 行不通。

Eval 测试的核心思路：**不测精确内容，测行为边界。**

- 用 `>=`，`<=`，`in` 代替 `==`
- 测试“有没有摘要”而非“摘要内容是什么”
- 正面 + 负面 + 边界 = 最小 Eval 集

步骤 1：用 AI 编程工具生成 eval_test.py

以下代码可以用 **OpenCode**、**Claude Code**、**Cursor**、**Trae** 或**通义灵码**等任意 AI 编程工具生成。

提示词：

```
请帮我在 v3-multi-agent/tests/ 下编写 eval_test.py，实现 AI 知识库的评估测试：

需求：
1. 使用 pytest 框架
2. 文件顶部用 dotenv 加载 .env，让 pytest 能读到 LLM_API_KEY；
   并屏蔽 PytestUnknownMarkWarning（避免自定义 slow 标记触发警告）
3. 定义 EVAL_CASES 列表，至少包含 3 种场景：
   - 正面案例：技术文章输入，预期有摘要、有关键词
   - 负面案例：无关内容输入，预期被过滤或标记为低相关
   - 边界案例：极短输入（如"AI"），预期不崩溃
4. 每个用例包含：name，input，expected（含检查函数或范围条件）
5. 测试函数使用范围断言（>=，<=，in），不用精确匹配
6. 包含一个 LLM-as-Judge 测试：
   - 让 LLM 对分析结果打分（1-10）
   - 断言分数 >= 5
7. 标记 LLM 测试为 pytest.mark.slow（可选跳过）
8. 包含一个不调用 LLM 的本地验证测试（验证 EVAL_CASES 结构）

依赖：workflows/model_client.py 的 chat() 函数
chat(prompt, system=...) 返回 (text, usage)
```

生成的代码（参考实现）：

```
"""Eval 评估测试 — AI 知识库质量验证

核心原则：
- 不测精确内容，测行为边界
- 用 >=, <=, in 代替 ==
- 正面 + 负面 + 边界 = 最小 Eval 集
- LLM-as-Judge 做质量评分
"""

import json
import os
import sys
import warnings

import pytest
from dotenv import load_dotenv

# 确保可以导入 workflows 模块
PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)
sys.path.insert(0, PROJECT_ROOT)

# 加载 .env，让 pytest 能读到 LLM_API_KEY
load_dotenv(os.path.join(PROJECT_ROOT, ".env"))

# 屏蔽 "Unknown pytest.mark.slow" 警告（slow 是自定义慢速标记）
warnings.filterwarnings("ignore", category=pytest.PytestUnknownMarkWarni

from workflows.model_client import chat


# ── 评估用例定义 ──────────────────────────────────────────

EVAL_CASES = [
    {
        "name": "正面案例 — 技术项目分析",
        "input": "LangGraph 是一个基于有向图的多 Agent 工作流编排框架，支持条件
        "expected": {
            "min_length": 50,
            "max_length": 1000,
            "must_contain_any": ["LangGraph", "工作流", "Agent", "图"],
        },
    },
    {
        "name": "负面案例 — 无关内容",
        "input": "今天天气真好，适合出去野餐，带上三明治和果汁。",
        "expected": {
            "max_length": 500,
```

```
            "should_mention_irrelevant": True,
        },
    },
    {
        "name": "边界案例 — 极短输入",
        "input": "AI",
        "expected": {
            "min_length": 1,
            "no_crash": True,
        },
    },
    {
        "name": "正面案例 — 英文技术内容",
        "input": "OpenAI released GPT-5 with 1M token context window and
        "expected": {
            "min_length": 30,
            "must_contain_any": ["GPT-5", "OpenAI", "token", "context"],
        },
    },
]


# ── 本地验证（不调 LLM）──────────────────────────────────

def test_eval_cases_structure():
    """验证 EVAL_CASES 的结构完整性（不消耗 token）"""
    assert len(EVAL_CASES) >= 3, "至少需要 3 个评估用例"

    names = [c["name"] for c in EVAL_CASES]
    assert any("正面" in n for n in names), "缺少正面案例"
    assert any("负面" in n for n in names), "缺少负面案例"
    assert any("边界" in n for n in names), "缺少边界案例"

    for case in EVAL_CASES:
        assert "name" in case, f"用例缺少 name 字段"
        assert "input" in case, f"用例 {case.get('name')} 缺少 input"
        assert "expected" in case, f"用例 {case.get('name')} 缺少 expecte


# ── LLM 评估测试（消耗 token）──────────────────────────────

@pytest.mark.slow
def test_eval_positive():
    """正面案例：技术内容应生成有意义的分析"""
    case = EVAL_CASES[0]
    prompt = f"请分析以下技术内容，输出 200 字以内的中文摘要：\n{case['input']}
    result, usage = chat(prompt, system="你是技术分析师。")

```

```
    expected = case["expected"]
    assert len(result) >= expected["min_length"], f"输出太短: {len(result
    assert len(result) <= expected["max_length"], f"输出太长: {len(result

    found = any(kw in result for kw in expected["must_contain_any"])
    assert found, f"输出应包含以下关键词之一: {expected['must_contain_any']}


@pytest.mark.slow
def test_eval_negative():
    """负面案例：无关内容应被识别"""
    case = EVAL_CASES[1]
    prompt = f"请判断以下内容是否与 AI 技术相关，如果不相关请说明：\n{case['inpu
    result, usage = chat(prompt, system="你是技术内容筛选器。")

    # 负面案例：输出应提及"不相关"或类似表述
    assert result is not None and len(result) > 0


@pytest.mark.slow
def test_eval_boundary():
    """边界案例：极短输入不应崩溃"""
    case = EVAL_CASES[2]
    try:
        result, usage = chat(f"请分析：{case['input']}", system="你是技术分
        assert result is not None
        assert len(result) > 0
    except Exception as e:
        pytest.fail(f"边界输入不应导致崩溃: {e}")


@pytest.mark.slow
def test_llm_as_judge():
    """LLM-as-Judge：让 LLM 对分析质量打分"""
    # 先生成分析结果
    analysis, _ = chat(
        "请分析 LangGraph 框架的核心优势和适用场景",
        system="你是技术分析师。输出 Markdown 格式。",
    )

    # 让另一个 LLM 调用评分
    judge_prompt = f"""请对以下技术分析的质量打分（1-10分）。

分析内容：
{analysis}

评分标准：
- 准确性：信息是否正确
```

```
- 深度：是否有洞察
- 实用性：读者能否据此行动

只返回一个数字（1-10），不要解释。"""

    score_text, _ = chat(judge_prompt, system="你是质量评审。只返回数字。", m

    try:
        score = int(score_text.strip())
    except ValueError:
        # 提取数字
        import re
        match = re.search(r"\d+", score_text)
        score = int(match.group()) if match else 5

    assert 1 <= score <= 10, f"评分应在 1-10 范围内，实际: {score}"
    assert score >= 5, f"分析质量评分过低: {score}/10"
    print(f"LLM-as-Judge 评分: {score}/10")


# ── 运行入口 ──────────────────────────────────────────────

if __name__ == "__main__":
    print("=== 本地验证（不消耗 token）===")
    test_eval_cases_structure()
    print(f"[OK] EVAL_CASES 结构验证通过，共 {len(EVAL_CASES)} 个用例")
    for c in EVAL_CASES:
        print(f"  - {c['name']}")

    print("\n提示：运行 LLM 测试请使用:")
    print("  pytest tests/eval_test.py -m slow -v")

```

步骤 2：理解代码

如果你对这段代码有疑问，可以让 AI 编程工具解释：

> `请解释 eval_test.py 的设计：`
> `1. 为什么用 must_contain_any 而不是 must_contain_all？`
> `2. pytest.mark.slow 有什么作用？`
> `3. LLM-as-Judge 的原理是什么？可靠吗？`
> `4. 为什么 test_eval_cases_structure 不调 LLM？`

关键设计解读：

设计点	为什么这样做
must_contain_any	LLM 输出不确定，只要包含相关词之一就算合格
pytest.mark.slow	LLM 测试消耗 token，标记后可选择性跳过
LLM-as-Judge	用 LLM 评判 LLM 的输出，比规则断言更灵活
本地验证	不消耗 token 就能验证测试框架本身是否正确

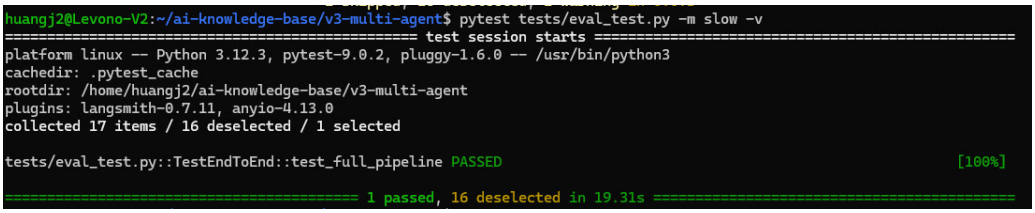
步骤 3：运行测试

```
cd ~/ai-knowledge-base

# 先跑本地验证（不消耗 token）
python tests/eval_test.py

# 再跑完整 Eval（消耗 token）
pytest tests/eval_test.py -m slow -v
```

期望输出（本地验证）：

```
=== 本地验证（不消耗 token）===
[OK] EVAL_CASES 结构验证通过，共 4 个用例
  - 正面案例 — 技术项目分析
  - 负面案例 — 无关内容
  - 边界案例 — 极短输入
  - 正面案例 — 英文技术内容

```

完整 Eval：



步骤 4：提交到 Git

```
cd ~/ai-knowledge-base/v3-multi-agent
git add tests/eval_test.py
git commit -m "feat: add eval test suite with positive/negative/boundary
```

完成！ Eval 测试覆盖了正面、负面、边界三种场景，加上 LLM-as-Judge 质量评分。范围断言适应 AI 输出的不确定性。